LESSON PLAN · PYTHON (GR 6–8) BEGINNER: PYTHON GAME CODERS

Lesson 7: Lists: Many Things in One Box

Module 2 | 60 minutes | lists, index, append, in | Grades 6–10 · ages 11–15

Objective: Make lists, read items by their index, add items with append, loop through a list and check what is in it.

Success criteria

- I can make a list and read items by their index.
- I can add items with append and count them with len.
- I can loop through a list with for.
- I can check if something is in a list with in.

Key vocabulary

- **list**: Many values stored in order inside [ ].
- **item**: One value inside a list.
- **index**: The position number of an item. The first is 0.
- **append**: Adds an item to the end of a list.
- **len()**: Counts the items in a list.
- **in**: Checks if a value is in a list: True or False.

You will need

- Projector with Kitabash (lesson video, Python Lab)
- Devices: 1 per learner, with a keyboard
- Printed worksheet (optional)
- Project files: team_list.py

Timed lesson plan (60 min)

Time	Stage	What happens
5 min	**Warm-up game: Human List**	Play Human List (unplugged): learners act as a list and respond to index, append and in calls.
5 min	**Hook and video**	Show the hook. Chat poll: “Will moves[0] print rock or paper?” Play the video with captions.
10 min	**Live demo and build-along**	Run the live demo, then everyone builds the cricket team list. Thumbs check: “Did len go up after append?”
8 min	**Challenge, predict and blanks**	Growing snack list, the index predict and list facts. Quick competition: fastest correct answer to team[1].
15 min	**Project in breakout pairs**	Pairs build Rock, Paper, Scissors. Each partner plays a full best-of-5 against the other’s game and tries an invalid move.
6 min	**Debug race**	Race to fix “the colour list”: three bugs. First to explain why index 3 crashed wins.

7 min	**Show-and-tell tournament**	Mini-tournament: three volunteers play their game live against the computer while the class cheers. Shout-out for the cleanest winner logic.
4 min	**Wrap-up and exit ticket**	Fist-to-five on the success criteria. Exit ticket: colours[1] and len(colours).

Unplugged warm-up

Human List (5 min game): five learners line up on camera (or hold up name cards) as a list. The class calls out “index 0!”, “index -1!”, “append Riya!” or “is Arjun in the list?” and the list must respond. Anyone who answers “index 1” as the first item gets a friendly buzzer.

Teaching the idea

A list stores many values in order in one variable. Items are numbered from 0, so the first item is list[0] and the last is list[-1]. append adds to the end and len counts the items. A for loop can visit every item directly, and the word in checks whether a value is in the list. Today learners use a list of moves with random.choice to build Rock, Paper, Scissors.

Analogy: A list is like a train: each coach holds one item, the coaches are numbered from 0 starting at the engine, and append adds a new coach at the back.

Live demo: 1) Type team = ["Aarav", "Meera", "Kabir"] and print(team). 2) Ask the class: “What is team[1]?” Take answers in the chat, then print it (Meera). 3) Print team[3] to show the IndexError, and explain the last index is 2; then show team[-1]. 4) Type team.append("Zoya") and print(len(team)). 5) Type for player in team: print("Go", player). 6) Type moves = ["rock", "paper", "scissors"], then player = input("Your move: ") and if player in moves: print("Good move!") else: print("Not a move!"). Run with rock and banana. 7) Add import random and print(random.choice(moves)).

Questions to ask

Ask	Listen for
What index does the first item have?	0.
How do you get the last item of any list?	list[-1].
What does team.append("Riya") do?	Adds Riya to the end of the list.
What does "banana" in moves give?	False.
Why did colours[3] crash a 3-item list?	Indexes go 0, 1, 2, so 3 does not exist.

Differentiation

Support
Give a printed index strip (0, 1, 2, 3 above boxes) so learners can write the items under their index numbers.

Stretch
Make Rock, Paper, Scissors, Lizard, Spock with 5 moves, or keep a history list of every move the player made and print it at the end.

Common misconceptions

Misconception	How to fix it
The first item is at index 1	Python counts from 0.
list.add() adds an item	Python lists use append().
for item in list gives the index numbers	It gives the items themselves.
"Rock" in moves is True when moves has "rock"	Capitals matter. Use .lower() on the input first.

Assessment

Use the Python Lab results (auto-checked), the quick checks and the Probe answers. Written answers arrive in your review queue.

Exit ticket
colours = ["red", "green", "blue"]. What is colours[1], and what is len(colours)?

Homework
Play your Rock, Paper, Scissors with two people at home and record the results. Then make a list of your 5 favourite foods and print them with a for loop.

Answer key — Kitabash activities

1. Python Lab: growing snack list (perform · python lab)

Goals checked: Adds the new snack · Counts with len() · Uses a for loop, append and len

One solution:

```
snacks = ["samosa", "jalebi", "pakora"]
new = input("Add a snack: ")
snacks.append(new)
for snack in snacks:
    print("-", snack)
print(f"We have {len(snacks)} snacks!")
```

2. Predict the output: index and length (perform · code output)

3. List facts (perform · fill blank)

Answers: square · 0 · append · in

4. Project: Rock, Paper, Scissors (perform · python lab)

Goals checked: Plays 5 rounds · Rejects moves that are not in the list · Invalid moves score nothing · Uses a list, random.choice and a for loop

Example program:

```
# Rock, Paper, Scissors: best of 5
# Made by: Aarav
import random

moves = ["rock", "paper", "scissors"]
player_score = 0
computer_score = 0

print("=== ROCK, PAPER, SCISSORS ===")
print("Best of 5 rounds. Type rock, paper or scissors.")

for round_num in range(1, 6):
    print(f"--- Round {round_num} ---")
    player = input("Your move: ").strip().lower()
    if player not in moves:
        print("Invalid move! Nobody scores this round.")
        continue
    computer = random.choice(moves)
    print(f"Computer chose {computer}")
    if player == computer:
        print("Draw!")
    elif (player == "rock" and computer == "scissors") or (player == "paper" and
computer == "rock") or (player == "scissors" and computer == "paper"):
        print("You win this round!")
        player_score += 1
    else:
        print("Computer wins this round!")
        computer_score += 1

print(f"Final score: You {player_score} - {computer_score} Computer")
if player_score > computer_score:
    print("You are the champion!")
elif player_score < computer_score:
    print("The computer wins this time.")
else:
    print("It's a tie!")
```

5. Debug it: the colour list (probe · python lab)

Goals checked: Prints the first colour · Prints the last colour · Adds yellow

One solution:

```
colours = ["red", "green", "blue"]
print("First:", colours[0])
print("Last:", colours[-1])
colours.append("yellow")
print(len(colours), "colours")
```

Answer key — worksheet

1. **Key words:** list = Many values stored in order inside [ ]. · item = One value inside a list. · index = The position number of an item. The first is 0. · append = Adds an item to the end of a list.
2. **Plan your program:** Goals: Adds the new snack; Counts with len(); Uses a for loop, append and len. One solution: snacks = ["samosa", "jalebi", "pakora"] new = input("Add a snack: ") snacks.append(new) for snack in snacks: print("-", snack) print(f"We have {len(snacks)} snacks!")
3. **Predict the output: index and length:** A. Meera 4
4. **List facts:** square · 0 · append · in
5. **Plan your program:** Goals: Prints the first colour; Prints the last colour; Adds yellow. One solution: colours = ["red", "green", "blue"] print("First:", colours[0]) print("Last:", colours[-1]) colours.append("yellow") print(len(colours), "colours")

Kitabash original curriculum.